

I2C Sensor - HTS221

Ankit Srivastava

EE2028

STM32L4S5xx Datasheet

STM32L4+ User Manual RM0432

B-L4S5I-IOT01A User Manual UM2708

HTS221 Datasheet

Acknowledgement: Slides adapted from Dr Rajesh Panicker



OVERVIEW

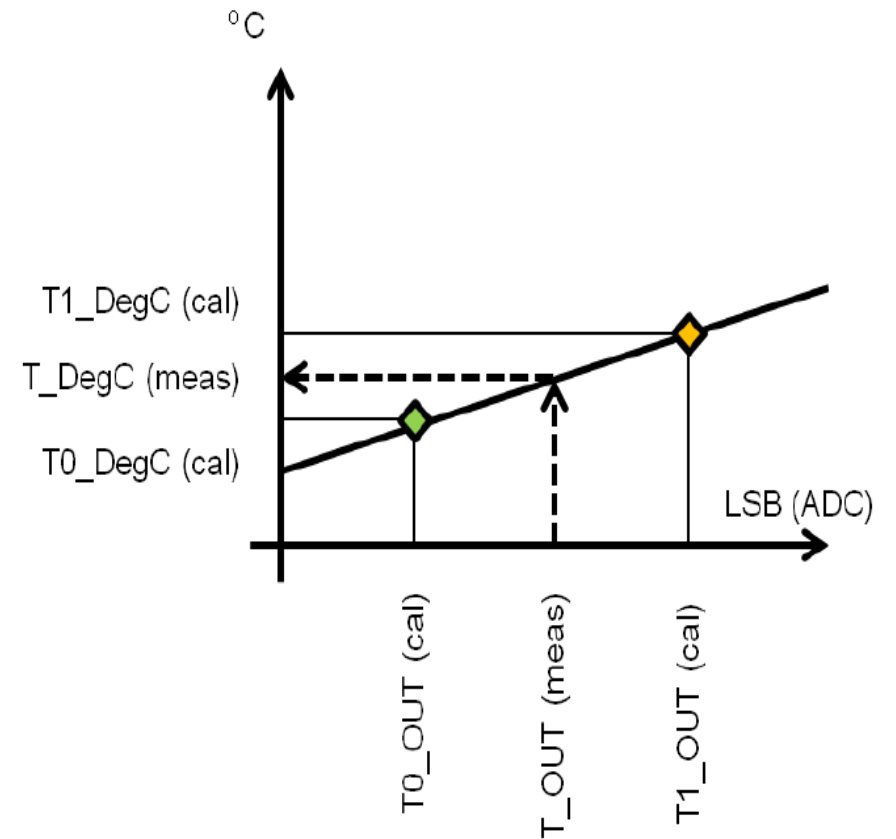
- HTS221 Sensor
- Registers: Control & Data
- HTS221 BSP code walkthrough
- Hardware abstraction

HTS221: TEMPERATURE SENSOR

- Measures 16-bit temperature and humidity
- 1 Hz to 12.5 Hz output data rate
- SPI and I2C interfaces supported
- Configured in I2C when CS pin is HIGH or unconnected
 - Standard mode and Fast mode
 - 7-bit slave address = 1011111 (0x5F)
 - Read address : 10111111 (0xBF)
 - Write address : 10111110 (0xBE)
 - Hard-coded by the manufacturer. Cannot be changed

HTS221: TEMPERATURE SENSOR

- Two different HTS221s could send different values for the same temperature (magnitude)!
- Make use of calibration data to find the actual temperature (magnitude)



HTS221 REGISTERS

WHO_AM_I (Device Identification, Address: 0x0F)

7	6	5	4	3	2	1	0
1	0	1	1	1	1	0	0

- This read-only register contains the device identifier, set to BCh
 - Value hard-coded by the manufacturer, need not be the same as slave address
- Read to ensure that we are talking to the correct device

Note: These register addresses are not 'offsets', but actual addresses local to the specific I2C Device.
Another I2C device can have a different register with the same address!

HTS221 REGISTERS

CTRL_REG1 (Control Register 1, Address: 0x20)

7	6	5	4	3	2	1	0
PD	Reserved				BDU	ODR1	ODR0

- PD: power down control. The device is in power-down mode when PD = 0 (default value after boot). The device is active when PD is set to 1
- BDU: block data update. 0 -> continuous update; 1 -> output registers not updated until MSB and LSB are both ready with the new value
 - Setting it to 1 prevents the reading of LSB and MSB related to different samples
 - Not to be set when reading only the MSB to get an approximate reading
- ODR [1:0] : output data rate selection (see table 17 in page 22)
 - BSP library sets it to 01 during initialization, configuring temp. measurement in 1 Hz mode

Register mapping

The table below provides a list of the 8-bit registers embedded in the device and the related addresses.

Table 15. Register address map

Name	Type	Register address (hex)	Default (hex)
Reserved		00-0E	Do not modify
WHO_AM_I	R	0F	BC
AV_CONF	R/W	10	1B
Reserved		11-1C	Do not modify
CTRL_REG1	R/W	20	0
CTRL_REG2	R/W	21	0
CTRL_REG3	R/W	22	0
Reserved		23-26	Do not modify
STATUS_REG	R	27	0
HUMIDITY_OUT_L	R	28	Output
HUMIDITY_OUT_H	R	29	Output
TEMP_OUT_L	R	2A	Output
TEMP_OUT_H	R	2B	Output
Reserved		2C-2F	Do not modify
CALIB_0..F	R/W	30-3F	Do not modify

Registers marked as *Reserved* must not be changed. Writing to those registers may cause permanent damage to the device.

HTS221 REGISTERS

TEMP_OUT_L (Temperature data [LSB], Address: 0x2A)

7	6	5	4	3	2	1	0
TOUT7	TOUT6	TOUT5	TOUT4	TOUT3	TOUT2	TOUT1	TOUT0

TEMP_OUT_H (Temperature data [MSB], Address: 0x2B)

15	14	13	12	11	10	9	8
TOUT15	TOUT14	TOUT13	TOUT12	TOUT11	TOUT10	TOUT9	TOUT8

- TOUT[15:0] is a signed (2's complement) value
- In HTS221, 7 LSB represents the actual register address while the MSB enables address autoincrement. If the MSB of the 8-bit register address field is '1', autoincrement is enabled
- Allows multiple sequential registers to be read/written in a single operation
- We can write (1<<7)|0x2A, i.e., 0xAA, and read two bytes in one transaction to get TOUT

HTS221 REGISTERS

Step 1: Temperature conversion from ADC_OUT (LSB) to °C

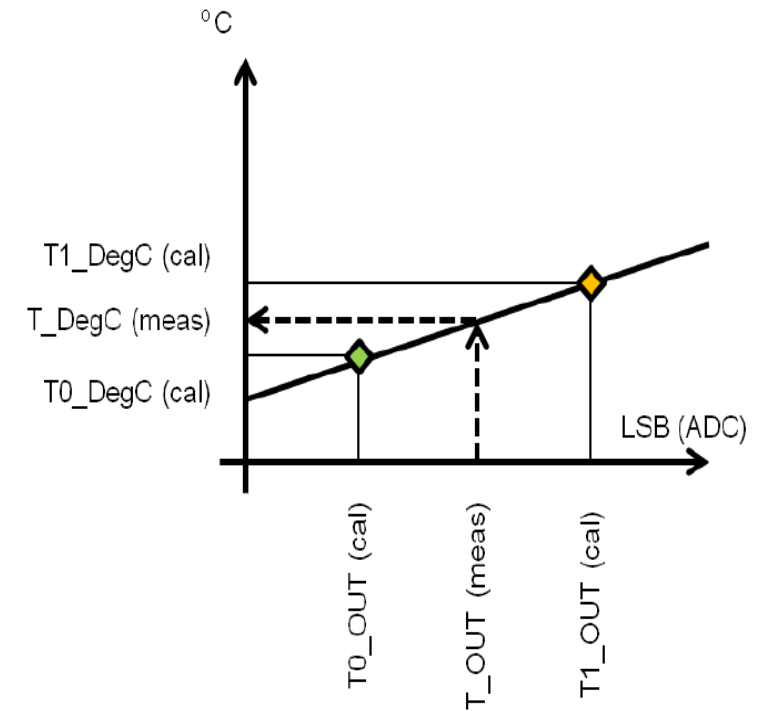
Data to build the Temperature calibration curve are stored in device registers.

Linear interpolation (example)

Input temperature LSB (ADC)	Output temperature (°C)
T0_OUT = 300	(Msb T0_degC U T0_degC)_x8 = 80 °C => 80/8 = 10.0°C
T1_OUT = 500	(Msb T1_degC U T1_degC)_x8 = 160 °C => 160/8 = 20.0°C

Temperature conversion:

T_OUT = 400	T_degC_x8 = 120 °C => T_degC = 120/8 = 15.0 °C
-------------	--



$$T_{actual} = T0_{degC} + (TOUT - T0OUT) \times (T1_{degC} - T0_{degC}) / (T1OUT - T0OUT)$$

- Temperature (magnitude) can be computed by linear interpolation of T_OUT value using calibration regs T0_OUT, T1_OUT, T0_degC_x8, T1_degC_x8 and T1/T0 msb regs
 - T0_OUT is the value (signed) corresponding to the temperature (magnitude) T0_DegC given by
 - $(T1/T0 \text{ msb}[1:0] \ll 8 \mid T0_degC_x8[7:0]) / 8 \text{ °C}$ (**'/8' approximated as >>3 in BSP library**)
 - T1_OUT is the value (signed) corresponding to the temperature (magnitude) T1_DegC given by
 - $(T1/T0 \text{ msb}[3:2] \ll 8 \mid T1_degC_x8[7:0]) / 8 \text{ °C}$

TEMPERATURE SENSOR CODE WALKTHROUGH

```
/*stm32l4s5i_iot01_tsensor.c*/
TSENSOR_DrvTypeDef *tsensor_drv;

uint32_t BSP_TSENSOR_Init(void)
{
    uint8_t ret = TSENSOR_ERROR;

#ifdef USE_LPS22HB_TEMP
    tsensor_drv = &LPS22HB_T_Drv;
#else /* USE_HTS221_TEMP */
    tsensor_drv = &HTS221_T_Drv;
#endif

    /* Very Low level (I2C) init */
    SENSOR_IO_Init();

    /* High level TSENSOR Init */
    tsensor_drv->Init(TSENSOR_I2C_ADDRESS, NULL);
    //The above has the same effect as the ones below
    //HTS221_T_Drv.Init(TSENSOR_I2C_ADDRESS, NULL);
    //HTS221_T_Init(TSENSOR_I2C_ADDRESS, NULL);
    ret = TSENSOR_OK;

    return ret;
}
```

```
/*stm32l4s5i_iot01.h*/
/* I2C Sensors address */
/* LPS22HB (Pressure) I2C Address */
#define LPS22HB_I2C_ADDRESS (uint8_t)0xBA
/* HTS221 (Humidity) I2C Address */
#define HTS221_I2C_ADDRESS (uint8_t)0xBE

#ifdef USE_LPS22HB_TEMP
/* LPS22HB Sensor hardware I2C address */
#define TSENSOR_I2C_ADDRESS LPS22HB_I2C_ADDRESS
#else /* USE_HTS221_TEMP */
/* HTS221 Sensor hardware I2C address */
#define TSENSOR_I2C_ADDRESS HTS221_I2C_ADDRESS
#endif
```

```
/*hts221.c*/
TSENSOR_DrvTypeDef HTS221_T_Drv =
{
    HTS221_T_Init,
    0,
    0,
    HTS221_T_ReadTemp
};
```

```
/*tsensor.h*/
typedef struct
{
    void (*Init)(uint16_t, TSENSOR_InitTypeDef *);
    uint8_t (*IsReady)(uint16_t, uint32_t);
    uint8_t (*ReadStatus)(uint16_t);
    float (*ReadTemp)(uint16_t);
}TSENSOR_DrvTypeDef;
```

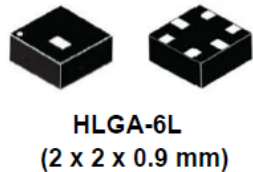
TEMPERATURE SENSOR CODE WALKTHROUGH



HTS221

Capacitive digital sensor for relative humidity and temperature

Datasheet - not recommended for new design



HLGA-6L
(2 x 2 x 0.9 mm)

Features

- 0 to 100% relative humidity range
- Supply voltage: 1.7 to 3.6 V
- Low power consumption: 2 μ A @ 1 Hz ODR
- Selectable ODR from 1 Hz to 12.5 Hz
- High rH sensitivity: 0.004% rH/LSB
- Humidity accuracy: $\pm 3.5\%$ rH, 20 to +80% rH
- Temperature accuracy: ± 0.5 $^{\circ}$ C, 15 to +40 $^{\circ}$ C
- Embedded 16-bit ADC
- 16-bit humidity and temperature output data
- SPI and I²C interfaces
- Factory calibrated
- Tiny 2 x 2 x 0.9 mm package

Description

HTS221 is in the process of being terminated and is not recommended for new design. The candidate replacement is SHT40-AD1B from Sensirion. A deep-dive transition guide, technical note TN1426, is available on www.st.com, providing a high-level reference to guide the user in replacing STMicroelectronics HTS221 sensor with the SHT4x sensor family from Sensirion as a high-quality alternative.

The HTS221 is an ultracompact sensor for relative humidity and temperature. It includes a sensing element and a mixed signal ASIC to provide the measurement information through digital serial interfaces.

The sensing element consists of a polymer dielectric planar capacitor structure capable of detecting relative humidity variations and is manufactured using a dedicated ST process.

The HTS221 is available in a small top-holed cap land grid array (HLGA) package guaranteed to operate over a temperature range from -40 $^{\circ}$ C to +120 $^{\circ}$ C.

```
/*hts221.c*/
TSENSOR_DrvTypeDef HTS221_T_Drv =
{
    HTS221_T_Init,
    0,
    0,
    HTS221_T_ReadTemp
};
```

```
/*lps22hb.c*/
TSENSOR_DrvTypeDef LPS22HB_T_Drv =
{
    LPS22HB_T_Init,
    0,
    0,
    LPS22HB_T_ReadTemp
};
```

TEMPERATURE SENSOR CODE WALKTHROUGH

```
/*hts221.c*/
void HTS221_T_Init(uint16_t DeviceAddr, TSensor_InitTypeDef *pInitStruct)
{
    uint8_t tmp;

    /* Read CTRL_REG1 */
    tmp = SENSOR_IO_Read(DeviceAddr, HTS221_CTRL_REG1); // #define HTS221_CTRL_REG1 (uint8_t)0x20
    // Reading CTRL_REG1 is necessary since bits marked as reserved should not be modified

    /* Enable BDU */
    tmp &= ~HTS221_BDU_MASK; // #define HTS221_BDU_MASK (uint8_t)0x04 // clear BDU aka bit 2
    tmp |= (1 << HTS221_BDU_BIT); // #define HTS221_BDU_BIT 2 // set BDU aka bit 2

    /* Set default ODR (Set ODR to 1Hz, ODR bits = 01) */
    tmp &= ~HTS221_ODR_MASK; // #define HTS221_ODR_MASK (uint8_t)0x03 // clear ODR aka bits 1, 0
    tmp |= (uint8_t)0x01; // ODR[1:0] set to 01

    /* Activate the device */
    tmp |= HTS221_PD_MASK; // Activate the device // set PD aka bit 7

    // tmp at this point is 0b1xxxx101, where x bits denotes reserved bits

    /* Apply settings to CTRL_REG1 */
    SENSOR_IO_Write(DeviceAddr, HTS221_CTRL_REG1, tmp);
}
```

Register mapping

The table below provides a list of the 8-bit registers embedded in the device and the related addresses.

Table 15. Register address map

Name	Type	Register address (hex)	Default (hex)
Reserved		00-0E	Do not modify
WHO_AM_I	R	0F	BC
AV_CONF	R/W	10	1B
Reserved		11-1C	Do not modify
CTRL_REG1	R/W	20	0
CTRL_REG2	R/W	21	0
CTRL_REG3	R/W	22	0
Reserved		23-26	Do not modify
STATUS_REG	R	27	0
HUMIDITY_OUT_L	R	28	Output
HUMIDITY_OUT_H	R	29	Output
TEMP_OUT_L	R	2A	Output
TEMP_OUT_H	R	2B	Output
Reserved		2C-2F	Do not modify
CALIB_0_F	R/W	30-3F	Do not modify

Registers marked as *Reserved* must not be changed. Writing to those registers may cause permanent damage to the device.

7	6	5	4	3	2	1	0
PD	Reserved			BDU	ODR1	ODR0	

CTRL_REG1 (Control Register 1, Address: 0x20)

TEMPERATURE SENSOR CODE WALKTHROUGH

```

/*lps22hb.c*/
static void LPS22HB_Init(uint16_t DeviceAddr)
{
    uint8_t tmp;

    /* Set Power mode */
    tmp = SENSOR_IO_Read(DeviceAddr, LPS22HB_RES_CONF_REG);

    tmp &= ~LPS22HB_LCEN_MASK;
    tmp |= (uint8_t)0x01; /* Set low current mode */

    SENSOR_IO_Write(DeviceAddr, LPS22HB_RES_CONF_REG, tmp);

    /* Read CTRL_REG1 */
    tmp = SENSOR_IO_Read(DeviceAddr, LPS22HB_CTRL_REG1);

    /* Set default ODR */
    tmp &= ~LPS22HB_ODR_MASK;
    tmp |= (uint8_t)0x30; /* Set ODR to 25Hz */

    /* Enable BDU */
    tmp &= ~LPS22HB_BDU_MASK;
    tmp |= ((uint8_t)0x02);

    /* Apply settings to CTRL_REG1 */
    SENSOR_IO_Write(DeviceAddr, LPS22HB_CTRL_REG1, tmp);
}

```

```

/*hts221.c*/
void HTS221_T_Init(uint16_t DeviceAddr, TSENSOR_InitTypeDef *pInitStruct)
{
    uint8_t tmp;

    /* Read CTRL_REG1 */
    tmp = SENSOR_IO_Read(DeviceAddr, HTS221_CTRL_REG1);

    /* Enable BDU */
    tmp &= ~HTS221_BDU_MASK;
    tmp |= (1 << HTS221_BDU_BIT);

    /* Set default ODR */
    tmp &= ~HTS221_ODR_MASK;
    tmp |= (uint8_t)0x01; /* Set ODR to 1Hz */

    /* Activate the device */
    tmp |= HTS221_PD_MASK;

    /* Apply settings to CTRL_REG1 */
    SENSOR_IO_Write(DeviceAddr, HTS221_CTRL_REG1, tmp);
}

```

```

/*lps22hb.c*/
void LPS22HB_T_Init(uint16_t DeviceAddr, TSENSOR_InitTypeDef *pInitStruct)
{
    LPS22HB_Init(DeviceAddr);
}

```

```

/*tsensor.h*/
typedef struct
{
    void (*Init)(uint16_t, TSENSOR_InitTypeDef *);
    uint8_t (*IsReady)(uint16_t, uint32_t);
    uint8_t (*ReadStatus)(uint16_t);
    float (*ReadTemp)(uint16_t);
}TSENSOR_DrvTypeDef;

```

```

/*hts221.c*/
TSENSOR_DrvTypeDef HTS221_T_Drv =
{
    HTS221_T_Init,
    0,
    0,
    HTS221_T_ReadTemp
};

```

```

/*lps22hb.c*/
TSENSOR_DrvTypeDef LPS22HB_T_Drv =
{
    LPS22HB_T_Init,
    0,
    0,
    LPS22HB_T_ReadTemp
};

```

TEMPERATURE SENSOR CODE WALKTHROUGH

SENSOR_IO_Read → Write of 1 byte followed by a read of 1 byte

- Write: Total 2 bytes to be sent over the I2C bus
 - 0xBE : slave address, to select HTS221 + W
 - 0x20 : to select HTS221_CTRL_REG1
- Read: Total 1 byte to be sent, and 1 byte to be received over the I2C bus
 - 0xBF : slave address, to select HTS221 + R
 - 0x?? : CTRL_REG1_content sent by the slave
- The exact sequence on the bus is
 - S > 0xBE > A > 0x20 > A > Sr > 0xBF > A > 0x?? > A' > P

SENSOR_IO_Write → A single write operation of 2 bytes

- Total 3 bytes to be sent over the I2C bus
 - 0xBE : slave address, to select HTS221 + W
 - 0x20 : to select HTS221_CTRL_REG1
 - 0b1xxxx101 : Content to be written into HTS221_CTRL_REG1
- The exact sequence on the bus is
 - S > 0xBE > A > 0x20 > A > 0b1xxxx101 > A > P

```
/*hts221.c*/
void HTS221_T_Init(uint16_t DeviceAddr, TSensor_InitTypeDef *pInitStruct)
{
    uint8_t tmp;

    /* Read CTRL_REG1 */
    tmp = SENSOR_IO_Read(DeviceAddr, HTS221_CTRL_REG1);

    /* Enable BDU */
    tmp &= ~HTS221_BDU_MASK;
    tmp |= (1 << HTS221_BDU_BIT);

    /* Set default ODR */
    tmp &= ~HTS221_ODR_MASK;
    tmp |= (uint8_t)0x01; /* Set ODR to 1Hz */

    /* Activate the device */
    tmp |= HTS221_PD_MASK;

    /* Apply settings to CTRL_REG1 */
    SENSOR_IO_Write(DeviceAddr, HTS221_CTRL_REG1, tmp);
}
```

Actions by slave in brown

Actions by master in blue



CONCLUSION

- HTS221 Sensor
- Registers: Control & Data
- HTS221 BSP code walkthrough
- Hardware abstraction